每个 kamacache 进程在哈希环中都是其中的一个节点，环中不同节点要互相通信，因此必须对外提供服务。

服务端模块

服务端模块是缓存系统对外提供服务的核心，负责接收和处理来自其他节点的请求

这里 kamacache 使用的是 grpc 来进行节点间的通信，同时集成了服务注册发现、健康检查和安全传输等特性。

核心结构设计

代码块

```go
// Server 定义缓存服务器
type Server struct {
    pb.UnimplementedLCacheServer
    addr       string           // 服务地址
    svcName    string           // 服务名称
    groups     *sync.Map        // 缓存组
    grpcServer *grpc.Server     // gRPC服务器
    etcdCli    *clientv3.Client // etcd客户端
    stopCh     chan error       // 停止信号
    opts       *ServerOptions   // 服务器选项
}
```

`Server` 结构体是服务器模块的核心，它包含以下关键字段：

- `UnimplementedLCacheServer`：gRPC 自动生成的基类，提供接口默认实现
- `addr`：服务监听地址，格式为 "IP:Port"
- `svcName`：服务名称，用于在服务注册中心标识服务
- `groups`：本地缓存组映射表，使用 `sync.Map` 保证并发安全
- `grpcServer`：gRPC 服务器实例
- `etcdCli`：etcd 客户端，用于服务注册
- `stopCh`：停止信号通道，用于服务优雅停止
- `opts`：服务器配置选项

服务创建和生命周期管理

代码块

```go
// NewServer 创建新的服务器实例
func NewServer(addr, svcName string, opts ...ServerOption) (*Server, error) {
    options := DefaultServerOptions
```

```go
    for _, opt := range opts {
        opt(options)
    }

    // 创建etcd客户端
    etcdCli, err := clientv3.New(clientv3.Config{
        Endpoints:   options.EtcdEndpoints,
        DialTimeout: options.DialTimeout,
    })
    if err != nil {
        return nil, fmt.Errorf("failed to create etcd client: %v", err)
    }

    // 创建gRPC服务器
    var serverOpts []grpc.ServerOption
    serverOpts = append(serverOpts, grpc.MaxRecvMsgSize(options.MaxMsgSize))

    if options.TLS {
        creds, err := loadTLSCredentials(options.CertFile, options.KeyFile)
        if err != nil {
            return nil, fmt.Errorf("failed to load TLS credentials: %v", err)
        }
        serverOpts = append(serverOpts, grpc.Creds(creds))
    }

    srv := &Server{
        addr:       addr,
        svcName:    svcName,
        groups:     &sync.Map{},
        grpcServer: grpc.NewServer(serverOpts...),
        etcdCli:    etcdCli,
        stopCh:     make(chan error),
        opts:       options,
    }

    // 注册服务
    pb.RegisterLCacheServer(srv.grpcServer, srv)

    // 注册健康检查服务
    healthServer := health.NewServer()
    healthpb.RegisterHealthServer(srv.grpcServer, healthServer)
    healthServer.SetServingStatus(svcName, healthpb.HealthCheckResponse_SERVING)

    return srv, nil
}

// Start 启动服务器
func (s *Server) Start() error {
    // 启动gRPC服务器
    lis, err := net.Listen("tcp", s.addr)
    if err != nil {
        return fmt.Errorf("failed to listen: %v", err)
    }

    // 注册到etcd
    stopCh := make(chan error)
    go func() {
        if err := registry.Register(s.svcName, s.addr, stopCh); err != nil {
            logrus.Errorf("failed to register service: %v", err)
            close(stopCh)
            return
        }
```

```
    }()

    logrus.Infof("Server starting at %s", s.addr)
    return s.grpcServer.Serve(lis)
}
```

服务创建流程包括以下步骤：

1. **应用配置选项**：合并默认选项和用户提供的选项
2. **创建 etcd 客户端**：用于服务注册
3. **配置 gRPC 服务器**：设置最大消息大小和可选的 TLS 加密
4. **创建服务器实例**：初始化 `Server` 结构体
5. **注册 gRPC 服务**：将缓存服务实现注册到 gRPC 服务器
6. **添加健康检查服务**：支持 gRPC 健康检查协议

服务启动过程：

1. **创建网络监听器**：在指定地址上监听 TCP 连接
2. **服务注册**：将服务信息注册到 etcd，使其对其他节点可见
3. **启动 gRPC 服务**：开始处理客户端请求

服务注册是异步执行的，避免阻塞主服务启动流程。

缓存操作接口

代码块

```
// Get 实现LCache服务的Get方法
func (s *Server) Get(ctx context.Context, req *pb.Request) (*pb.ResponseForGet, error) {
    group := GetGroup(req.Group)
    if group == nil {
        return nil, fmt.Errorf("group %s not found", req.Group)
    }

    view, err := group.Get(ctx, req.Key)
    if err != nil {
        return nil, err
    }

    return &pb.ResponseForGet{Value: view.ByteSLice()}, nil
}

// Set 实现LCache服务的Set方法
func (s *Server) Set(ctx context.Context, req *pb.Request) (*pb.ResponseForGet, error) {
    group := GetGroup(req.Group)
    if group == nil {
        return nil, fmt.Errorf("group %s not found", req.Group)
    }

    // 从 context 中获取标记，如果没有则创建新的 context
    fromPeer := ctx.Value("from_peer")
    if fromPeer == nil {
        ctx = context.WithValue(ctx, "from_peer", true)
```

```
    }

    if err := group.Set(ctx, req.Key, req.Value); err != nil {
        return nil, err
    }

    return &pb.ResponseForGet{Value: req.Value}, nil
}
```

获取缓存的处理流程：

1. **查找缓存组**：根据请求中的组名找到对应的缓存组
2. **获取缓存值**：调用缓存组的 Get 方法获取数据
3. **构造响应**：将获取的数据封装到响应中返回

设置缓存的处理流程：

1. **查找缓存组**：根据请求中的组名找到对应的缓存组
2. **标记来源**：在上下文中添加标记，表明请求来自对等节点，避免循环传播
3. **设置缓存值**：调用缓存组的 Set 方法存储数据
4. **构造响应**：将原始值封装到响应中返回

这里的 `fromPeer` 标记很重要，它防止了缓存更新无限传播的问题，是分布式缓存系统中的关键设计。

节点选择器

采用一致性哈希算法进行节点选择，结合 etcd 实现动态服务发现，是连接客户端和服务端的桥梁。

本模块解决了分布式缓存中的核心问题：

1. 如何在多个缓存节点之间均匀分布数据
2. 如何动态感知节点的加入和退出
3. 如何在节点变化时最小化数据迁移
4. 如何选择合适的节点处理特定的缓存请求

核心接口设计

代码块

```
// PeerPicker 定义了peer选择器的接口
type PeerPicker interface {
    PickPeer(key string) (peer Peer, ok bool, self bool)
    Close() error
}
```

`PeerPicker` 是节点选择器的抽象接口，定义了两个核心方法：

- `PickPeer`：根据键选择合适的缓存节点，返回节点实例、是否找到、是否为本地节点
- `Close`：关闭选择器并释放资源

这种抽象设计使得节点选择策略可以灵活更换，例如可以实现基于地理位置、负载情况或其他策略的选择器。

代码块

```go
// Peer 定义了缓存节点的接口
type Peer interface {
    Get(group string, key string) ([]byte, error)
    Set(ctx context.Context, group string, key string, value []byte) error
    Delete(group string, key string) (bool, error)
    Close() error
}
```

`Peer` 接口定义了缓存节点的基本操作：

- `Get`：获取缓存数据
- `Set`：设置缓存数据
- `Delete`：删除缓存数据
- `Close`：关闭节点连接并释放资源

该接口是对远程缓存节点操作的抽象，隐藏了底层通信细节，使得上层应用可以用统一的方式操作本地或远程缓存。

代码块

```go
// ClientPicker 实现了PeerPicker接口
type ClientPicker struct {
    selfAddr string
    svcName  string
    mu       sync.RWMutex
    consHash *consistenthash.Map
    clients  map[string]*Client
    etcdCli  *clientv3.Client
    ctx      context.Context
    cancel   context.CancelFunc
}
```

`ClientPicker` 是 `PeerPicker` 接口的核心实现，包含以下关键字段：

- `selfAddr`：当前节点的地址，用于识别自身
- `svcName`：服务名称，用于服务发现
- `mu`：读写互斥锁，保证并发安全
- `consHash`：一致性哈希实现，用于节点选择
- `clients`：节点地址到客户端的映射
- `etcdCli`：etcd 客户端，用于服务发现
- `ctx` 和 `cancel`：上下文和取消函数，用于控制生命周期

这种设计将节点选择、客户端管理和服务发现集成在一起，形成一个完整的节点管理解决方案。

服务发现与节点管理

代码块

```
// startServiceDiscovery 启动服务发现
func (p *ClientPicker) startServiceDiscovery() error {
    // 先进行全量更新
    if err := p.fetchAllServices(); err != nil {
        return err
    }

    // 启动增量更新
    go p.watchServiceChanges()
    return nil
}
```

服务发现分为两个阶段：

1. **全量更新**：获取当前所有可用的服务节点
2. **增量更新**：监听节点变化并实时更新

这种设计确保了选择器能够立即获取当前的完整节点列表，并在后续及时感知节点的变化

一致性哈希与节点选择

代码块

```
// PickPeer 选择peer节点
func (p *ClientPicker) PickPeer(key string) (Peer, bool, bool) {
    p.mu.RLock()
    defer p.mu.RUnlock()

    if addr := p.consHash.Get(key); addr != "" {
        if client, ok := p.clients[addr]; ok {
            return client, true, addr == p.selfAddr
        }
    }
    return nil, false, false
}
```

节点选择流程：

1. 获取读锁，确保并发安全
2. 使用一致性哈希算法根据键选择地址
3. 查找对应地址的客户端
4. 返回客户端、是否找到、是否为本地节点

这个实现的返回值包括三个部分：

- **客户端实例**：用于进行后续操作
- **是否找到节点**：表示选择是否成功
- **是否为本地节点**：区分本地操作和远程操作